



Conectar un sitio web a
una base de datos
utilizando PDO

PDO: Objetos de Datos de PHP

- ▶ La extensión *Objetos de Datos de PHP* define una interfaz para poder acceder a bases de datos en PHP.
- ▶ PDO proporciona una capa de abstracción de *acceso a datos*, lo que significa que, independientemente del gestor de bases de datos que se esté utilizando, se emplean las mismas funciones para realizar consultas y obtener datos.
- ▶ PDO permite acceder a diferentes sistemas gestores de bases de datos **con un controlador específico** (MySQL, MariaDB, SQLite, Oracle...) mediante el cual se conecta.

Ejecuta `phpinfo()` para saber que controladores están instalados en tu servidor web.

PDO: Objetos de Datos de PHP

- ▶ Considerad PDO como un conjunto de clases que vienen empaquetadas con PHP para facilitarle la interacción con su base de datos.
- ▶ Cuando desarrollamos una aplicación PHP que trabaje con una Base de datos necesitamos resolver diferentes problemas:
 - Establecer una conexión con la BD
 - Crear una consulta
 - Utilizar el recurso de conversión de resultados devueltos por la BD en un array asociativo u otra estructura de datos.
 - Evitar la inyección SQL, utilizando **sentencias preparadas**.

Inyección SQL

- ▶ La inyección SQL, o SQLi, es un tipo de ataque a una aplicación web que permite a un atacante insertar sentencias SQL maliciosas en la aplicación web, obteniendo potencialmente acceso a datos sensibles en la base de datos o destruyendo estos datos, la inyección SQL fue descubierta por primera vez por Jeff Forristal en 1998.
- ▶ En las dos décadas que han transcurrido desde su descubrimiento, la inyección SQL ha sido sistemáticamente la principal prioridad de los desarrolladores web a la hora de crear aplicaciones.

`select * from usuarios where nombre='luis' and clave='$variable';`

- ▶ Imagina la típica aplicación web que implica solicitudes de bases de datos a través de las entradas del usuario y la entrada se realiza a través de un formulario, donde pide usuario y clave.
- ▶ El usuario introduce en el campo **clave** del formulario: **`'98756' or '1=1'`**
- ▶ La instrucción sql quedaría:

`select * from usuarios where nombre='luis' and clave='98756' or '1=1';`

PDO: Objetos de Datos de PHP

El sistema PDO se fundamenta en 3 clases: **PDO**, **PDOStatement** y **PDOException**:

- ▶ La **clase PDO** se encarga de mantener la conexión a la base de datos y otro tipo de conexiones específicas como transacciones, además de crear instancias de la clase PDOStatement.
- ▶ La **clase PDOStatement**, es la que maneja las sentencias SQL y devuelve los resultados.
- ▶ La **clase PDOException** se utiliza para manejar los errores.

Clase PDO

Clase PDO

Representa **una conexión entre PHP y un servidor de bases de datos.**

► Crea un objeto de la clase PDO para representar una conexión a la base de datos solicitada.

PDO::__construct

```
public PDO::__construct(  
    string $dsn,  
    ?string $username = null,  
    ?string $password = null,  
    ?array $options = null )
```

Clase PDO

`string $dsn`

un DSN consiste en el nombre del controlador de PDO, en nuestro caso `mysql`, seguido por dos puntos, y a continuación la sintaxis específica del controlador de PDO para la conexión:

```
$dsn = 'mysql:host=localhost;dbname=prueba1;port=3306;charset=utf8';
```


Clase PDO

Ejemplo de conexión

```
const HOST='localhost';
const DATABASE='prueba1';
const USERNAME='root';
const PASSWORD='';
const PORT='3306';//puerto en el que escucha MariaDB
const CHARSET='UTF8';
$conexion = new PDO('mysql:host='.HOST.';dbname='.DATABASE.';port='.PORT.';charset='.CHARSET, USERNAME, PASSWORD);
```

Clase PDOException

Excepciones

El método `setAttribute()` de la clase PDO se utiliza para establecer atributos de la conexión a la base de datos. Este método permite modificar el comportamiento del objeto PDO en tiempo de ejecución y se utiliza para configurar el manejo de errores, entre otras opciones

```
public PDO::setAttribute(int $attribute, mixed $value): bool
```

- **\$attribute**: Es el atributo que deseas establecer. Existen varios atributos predefinidos que puedes usar, como `PDO::ATTR_ERRMODE`, `PDO::ATTR_DEFAULT_FETCH_MODE`, entre otros.
- **\$value**: Es el valor que deseas asignar al atributo especificado.

PDO::ATTR_ERRMODE y PDO::ERRMODE_EXCEPTION

Cuando se establece el atributo PDO::ATTR_ERRMODE con el valor PDO::ERRMODE_EXCEPTION, **se indica que deseas que PDO lance una excepción** PDOException cada vez que ocurre un error en la operación de la base de datos.

Manejo de Errores: Al lanzar excepciones en lugar de solo devolver códigos de error, puedes manejar los errores de manera más efectiva mediante bloques try-catch. Esto te permite capturar detalles sobre el error, como el mensaje de error, el código de error, el fichero del error..., lo que es útil para depuración.

Seguridad: Al utilizar excepciones, evitas exponer información sensible sobre la estructura de la base de datos. Cuando un error ocurre, puedes personalizar cómo se informa y se registra el error, evitando mostrar información que podría ser utilizada por un atacante.

```
<?php
// Conexión a la base de datos
const HOST = 'localhost';
const DATABASE = 'probando';
const USERNAME = 'root';
const PASSWORD = '';
const PORT = '3306'; //puerto en el que escucha MariaDB
const CHARSET = 'UTF8';

try {
    $dns = "mysql:host=" . HOST . ";dbname=" . DATABASE . ";port=" . PORT . ";charset=" . CHARSET;

    // $conexion contiene el objeto PDO que representa la conexión a la base de datos
    $conexion = new PDO($dns, USERNAME, PASSWORD);

    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {

    echo "<pre>Error: " . $e->getMessage() . "<br>";
    echo "Código de error: " . $e->getCode() . "<br>";
    echo "Archivo: " . $e->getFile() . "<br>";
    echo "Línea: " . $e->getLine() . "<br>";
    echo $cadena = print_r($e->getTrace(), true); //Array errores, convertido a string para visualizarlo
    echo "Rastro: " . $e->getTraceAsString(). "</pre>";

    // Archivo: El nombre del archivo donde ocurrió cada llamada.
    // Línea: El número de línea en el archivo donde ocurrió cada llamada.
    // Función: El nombre de la función o método que fue llamado.
    // Argumentos: Los argumentos que fueron pasados a la función o método
    exit;
}
```

```

$logFile = "miFicheroErrores.log";
// Eliminar el archivo de log si existe, para que no se acumulen los errores
if (file_exists($logFile)) {
    unlink($logFile);
}
//$conexion = new PDO('mysql:host=' . HOST . ';dbname=' . DATABASE . ';port=' . PORT . ';charset=' . CHARSET, USERNAME, PASSWORD);
try {
    $dns = "mysql:host=" . HOST . ";dbname=" . DATABASE . ";port=" . PORT . ";charset=" . CHARSET;
    // $conexion contiene el objeto PDO que representa la conexión a la base de datos
    $conexion = new PDO($dns, USERNAME, PASSWORD);
    // Establecer el modo de error de PDO a excepción
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    // Consulta de prueba
    $sql = "SELECT * FROM usuarios";
    // Ejecutar la consulta. $resultado contiene el objeto PDOStatement con los resultados
    $resultado = $conexion->query($sql);
    // Obtener los resultados
    $usuarios = $resultado->fetchAll(PDO::FETCH_ASSOC);
    var_dump($usuarios);
} catch (PDOException $e) {
    echo "<pre>Error: " . $e->getMessage() . "<br>";
    echo "Código de error: " . $e->getCode() . "<br>";
    echo "Archivo: " . $e->getFile() . "<br>";
    echo "Línea: " . $e->getLine() . "<br>";
    echo "Rastro: " . $e->getTraceAsString() . "</pre>";
    // Rastro: Una lista de todas las llamadas que se hicieron para llegar a la excepción.
    // Archivo: El nombre del archivo donde ocurrió cada llamada.
    // Línea: El número de línea en el archivo donde ocurrió cada llamada.
    // Función: El nombre de la función o método que fue llamado.
    // Argumentos: Los argumentos que fueron pasados a la función o método

    // Registrar los errores en el archivo de log, creamos un string con el mensaje que enviaremos al fichero
    $detallesError = "Error: " . $e->getMessage() . "\n" .
        "Código de error: " . $e->getCode() . "\n" .
        "Archivo: " . $e->getFile() . "\n" .
        "Línea: " . $e->getLine() . "\n";
    // funcion error_log() escribe un mensaje de error en el log del servidor, 3 indica que el mensaje de error se escribirá en un archivo
    error_log($detallesError, 3, "miFicheroErrores.log");
    exit;
} finally {
    // $resultado es un objeto PDOStatement que contiene los resultados de la consulta. Cursor es el nombre que recibe.
    // $conexion es un objeto PDO que representa la conexión a la base de datos
    // Cerrar el cursor, liberar recursos y evitar bloqueos
    $resultado->closeCursor();
    // Cerrar la conexión
    $conexion = null;
}

```

Métodos para Ejecutar Instrucciones SQL en PDO

En la clase PDO, existen dos métodos principales para ejecutar instrucciones SQL: query() y prepare(). **Ambos métodos devuelven un objeto de la clase PDOStatement**, que permite trabajar con los resultados de las consultas.

```
public query ( string $statement ) : PDOStatement
```

```
public prepare ( string $statement [,array $driver_options = array()]) : PDOStatement
```

Métodos para Ejecutar Instrucciones SQL en PDO

```
public query(string $statement) : PDOStatement
```

Descripción: Este método se utiliza para ejecutar una consulta SQL directamente. Es adecuado consultas simples que no requieren parámetros de entrada, como las instrucciones SELECT que no necesitan ser parametrizadas.

Uso: Cuando llamas a query(), no necesitas preparar la consulta de antemano. Es más simple y directo, pero **no protege contra inyecciones SQL** si se utilizan datos del usuario.

Métodos para Ejecutar Instrucciones SQL en PDO

```
public prepare(string $statement[, array $driver_options = array()]) : PDOStatement
```

Descripción: Este método se utiliza para preparar una consulta SQL. Es más flexible y seguro, ya que permite el uso de parámetros que pueden ser vinculados a valores de manera segura. Esto es crucial para prevenir inyecciones SQL.

Uso: Cuando usas `prepare()`, debes ejecutar la consulta usando el método `execute()`, que puede aceptar un array de parámetros. Se recomienda para consultas más complejas y cuando se manipulan datos que provienen de usuarios.

Muy importante

Solo debemos utilizar **query()** en consultas simples sin condicionales ni variables, es muy inseguro y permite inyección SQL.

Ejemplo: obtener un objeto PDOStatement

```
$conexion = new PDO('mysql:host='.HOST.';dbname='.DATABASE.';port='.PORT.';charset='.CHARSET, USERNAME, PASSWORD);

$conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$sql='select id, nombre, clave from usuarios';

$enlaceDatos = $conexion->query($sql);//$enlaceDatos es un objeto de PDOStatement
```

Clase PDOStatement

CLASE PDOStatement

Esta clase es la que maneja las sentencias SQL y devuelve los resultados.

► La consulta de datos se realiza mediante los métodos:

`PDOStatement::fetch`

- método que obtienen la siguiente fila de un conjunto de resultados

`PDOStatement::fetchAll`

- devuelve todos los resultados en un solo array.

Antes o durante la llamada a `fetch()` o `fetchAll()` hay que especificar cómo se quieren devolver los datos.

CLASE PDOStatement

Formato de los datos devueltos por la base de datos

PDO::FETCH_BOTH: el valor por defecto. Devuelve un array indexado cuyos índices son tanto el **nombre de las columnas** como **números**, la primera columna de la tabla se numera con 0.

PDO::FETCH_ASSOC: devuelve un array indexado cuyos índices son el **nombre de las columnas**.

PDO::FETCH_NUM: devuelve un array indexado cuyos índices son **números que se corresponden con las columnas**.

PDO::FETCH_OBJ: devuelve un objeto anónimo con nombres de atributos o propiedades que corresponden con los nombres de las columnas.

```
<?php

const HOST='localhost';
const DATABASE='probando';
const USERNAME='root';
const PASSWORD='';
const PORT='3306';//puerto en el que escucha MariaDB
const CHARSET='UTF8';

try{
// $dsn ='mysql:host=localhost;dbname=prueba1;port=3306;charset=utf8';
    $conexion = new PDO('mysql:host='.HOST.';dbname='.DATABASE.';port='.PORT.';charset='.CHARSET, USERNAME, PASSWORD);
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql=SELECT id, nombre, email FROM usuarios';
    $enlaceDatos = $conexion->query($sql);

    while ($fila = $enlaceDatos->fetch(PDO::FETCH_ASSOC)) {
        echo $fila['id'] . "<br>";
        echo $fila['nombre'] . "<br>";
        echo $fila['email'] . "<br>";
    }
}catch (PDOException $e){
    echo " <pre><br>Error: " . $e->getMessage();
    echo $cadena = print_r($e->getTrace(), true). "</pre>";
    exit();
} finally {
    // Cerrar el cursor, liberar recursos y evitar bloqueos
    $enlaceDatos ->closeCursor();
    // Cerrar la conexión
    $conexion = null;
}

?>
```

`bindParam()`

`bindValue()`

Consultas preparadas

- ▶ Muchas de las bases de datos admiten el concepto de sentencias preparadas o parametrizadas.
- ▶ Una sentencia preparada es una plantilla de SQL que las aplicaciones quieren ejecutar, y pueden ser personalizadas utilizando parámetros variables.

- ▶ `select nombre from prueba where clave='$campo1'`
- ▶ `select nombre from prueba where clave=?`
- ▶ `select nombre from prueba where clave=:variable`

- ▶ Las sentencias preparadas ofrecen dos grandes beneficios:
 1. La consulta sólo necesita ser analizada (o preparada) una vez, pero puede ser ejecutada muchas veces con diferentes parámetros.
 2. Los parámetros para las sentencias preparadas **no necesitan estar entrecomillados**; el controlador automáticamente se encarga de esto. Si una aplicación usa exclusivamente sentencias preparadas, el desarrollador puede estar seguro de que **no hay cabida para inyecciones de SQL** (sin embargo, si otras partes de la consulta se construyen con datos de entrada sin escapar, aún es posible que ocurran ataques de inyecciones de SQL).



Las funciones `bindParam` y `bindValue` en PDO se utilizan para asociar valores a los parámetros en una sentencia SQL preparada.

Aunque ambas funciones parecen similares, tienen diferencias importantes en su comportamiento y uso.

► `bindParam()`

- **Asociación por Referencia:** `bindParam()` asocia el parámetro a una variable por referencia. Esto significa que el valor de la variable se evalúa en el momento de la ejecución de la sentencia, no en el momento de la llamada a `bindParam()`.
- **Uso Típico:** Se utiliza cuando necesitas asociar una variable cuyo valor puede cambiar antes de la ejecución de la sentencia.

► `bindValue()`

- **Asociación por Valor:** `bindValue()` asocia el parámetro a un valor específico en el momento de la llamada. El valor no puede cambiar después de la llamada a `bindValue()`.
- **Uso Típico:** Se utiliza cuando tienes un valor fijo que no cambiará antes de la ejecución de la sentencia.

```
// Ejemplo con bindParam()
```

```
$sql = "SELECT * FROM usuarios WHERE id = :id";
```

```
$stmt = $conexion->prepare($sql);
```

```
$id = 1;
```

```
$stmt->bindParam(':id', $id, PDO::PARAM_INT);
```

```
$id = 2; // Cambiamos el valor de $id antes de ejecutar
```

```
$stmt->execute();
```

```
$usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
var_dump($usuarios);
```

```
// Ejemplo con bindValue()
```

```
$sql = "SELECT * FROM usuarios WHERE id = :id";
```

```
$stmt = $conexion->prepare($sql);
```

```
$id = 1;
```

```
$stmt->bindValue(':id', $id, PDO::PARAM_INT);
```

```
$id = 2; // Cambiamos el valor de $id, pero no afecta a la sentencia preparada
```

```
$stmt->execute();
```

```
$usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
var_dump($usuarios);
```

```
try {
    $conexion = new PDO($dsn, $usuario, $contrasena);
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    //Consulta preparada. Instrucción SQL INSERT para la tabla usuarios
    $sqlInsert = 'INSERT INTO usuarios (nombre, clave) VALUES (:nombre, :clave)';
    $stmtInsert = $conexion->prepare($sqlInsert);

    // Valores de los parámetros
    $nombre = 'Pepito';
    $clave = '963';

    // Asignar valores a los parámetros y especificar los tipos de datos
    $stmtInsert->bindParam(':nombre', $nombre, PDO::PARAM_STR);
    $stmtInsert->bindParam(':clave', $clave, PDO::PARAM_STR);

    // Ejecutar la consulta
    $stmtInsert->execute();
} catch (PDOException $e) {
    echo "<br>Error: " . $e->getMessage();
    echo "<br>Línea del error: " . $e->getLine();
    echo "<br>Archivo del error: " . $e->getFile();
}
```

```
try {
    $conexion = new PDO($dsn, $usuario, $contrasena);
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    // Instrucción SQL INSERT para usuarios enviando un array al método execute()

    $sqlInsert = 'INSERT INTO usuarios (nombre, clave) VALUES (:nombre, :clave)';
    $stmtInsert = $conexion->prepare($sqlInsert);
    //NO puedo especificar los tipos de datos
    //No es necesario incluir bindParam y es una consulta preparada
    $datos = ['nombre' => 'Pepito2', 'clave' => '852'];
    $stmtInsert->execute($datos);
} catch (PDOException $e) {
    echo "<br>Error: " . $e->getMessage();
    echo "<br>Línea del error: " . $e->getLine();
    echo "<br>Archivo del error: " . $e->getFile();
}
```

```
try {
    $conexion = new PDO($dsn, $usuario, $contrasena);
    $conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sqlInsert = 'INSERT INTO usuarios (nombre, clave) VALUES (:nombre, :clave)';
    $stmtInsert = $conexion->prepare($sqlInsert);

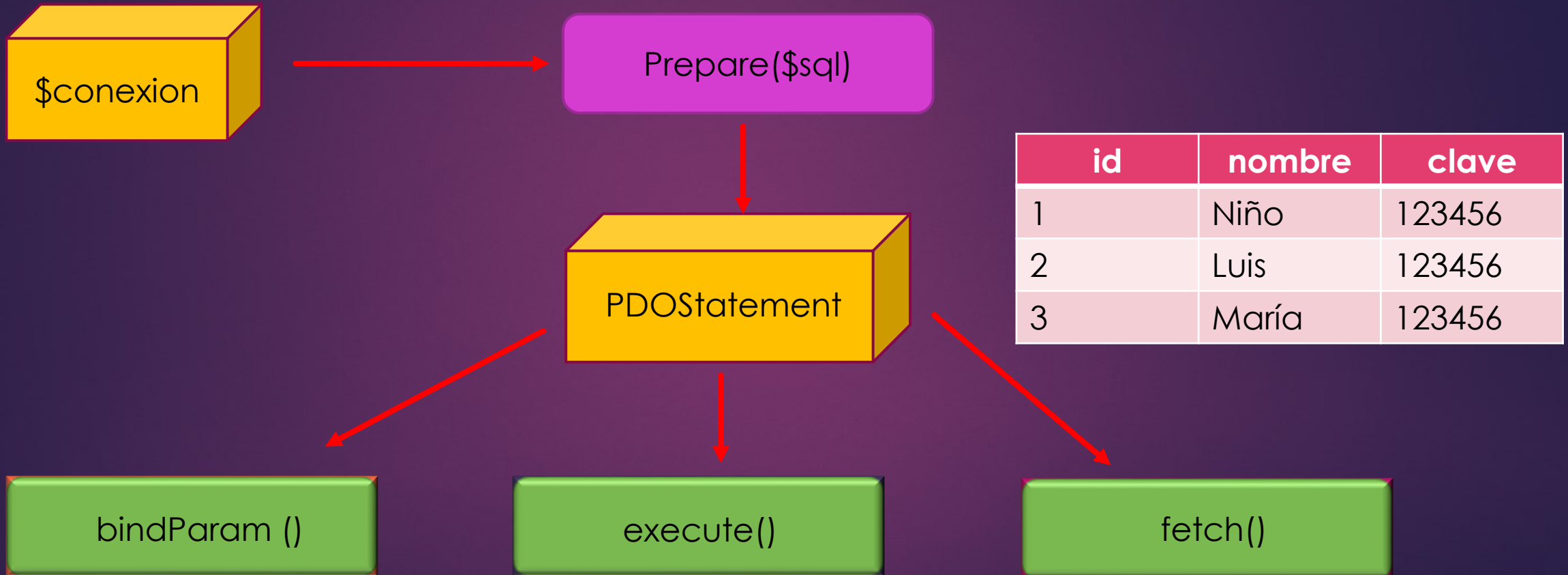
    $usuarios = [
        ['nombre' => 'Usuario1', 'clave' => 'Clave1'],
        ['nombre' => 'Usuario2', 'clave' => 'Clave2'],
        ['nombre' => 'Usuario3', 'clave' => 'Clave3'],
        ['nombre' => 'Usuario4', 'clave' => 'Clave4'],
        ['nombre' => 'Usuario5', 'clave' => 'Clave5']
    ];

    foreach ($usuarios as $usuario) {
        $stmtInsert->bindParam(':nombre', $usuario['nombre'], PDO::PARAM_STR);
        $stmtInsert->bindParam(':clave', $usuario['clave'], PDO::PARAM_STR);
        $stmtInsert->execute();
    }

    //Otra opción: No puedo indicar los tipos de datos, pero no es necesario incluir bindParam()
    foreach ($usuarios as $usuario) {
        $stmtInsert->execute($usuario);
    }
} catch (PDOException $e) {
    echo "<br>Error:{$e->getMessage()}<br>Línea del error: {$e->getLine()} <br>Archivo del error: {$e->getFile()} " ;
}
```

```
$conexion = new PDO('mysql:host='.HOST.';dbname='.DATABASE.';port='.PORT.';charset='.CHARSET, USERNAME, PASSWORD);
```

```
$sql="select id, nombre, clave from usuarios where clave=? and id>? ";
```



Consultas preparadas o parametrizadas con PDO

- La clase PDOStatement es la que trata las sentencias SQL.
- Una instancia de PDOStatement **se crea** cuando se llama a `PDO->prepare()`
- Con ese objeto creado se llama a métodos como `bindParam()` para pasar valores o `execute()` para ejecutar sentencias.
- PDO facilita el uso de sentencias preparadas en PHP, que mejoran el rendimiento y la seguridad de la aplicación.
- Cuando se obtienen, seleccionan, insertan, borran o actualizan datos, **el esquema es**:

PREPARE -> [BIND] -> EXECUTE-> [FETCH]

Consultas preparadas o parametrizadas con PDO

- Se pueden indicar los parámetros en la sentencia sql preparada de dos formas:

1. Mediante un interrogante ?

```
$sql="select id, nombre, clave from usuarios where clave=? and id=? ";
```

2. Con un nombre específico :nombre

```
$sql="INSERT INTO clientes (nombre, ciudad) VALUES (:nombre, :ciudad)";
```

Consultas preparadas o parametrizadas con PDO

Utilizando interrogantes para los valores

```
//Prepare
$enlace_datos = $conexion->prepare("INSERT INTO Clientes (nombre, ciudad) VALUES (?, ?)");
// Bind
$nombre = "Peter";
$ciudad = "Madrid";
$enlace_datos->bindParam(1, $nombre);
$enlace_datos->bindParam(2, $ciudad);
// Execute
$enlace_datos->execute();
```

Consultas preparadas o parametrizadas con PDO

Utilizando un nombre específico para las variables

```
// Prepare
$enlace_datos = $conexion->prepare("INSERT INTO Clientes (nombre, ciudad) VALUES (:nombre, :ciudad)");

// Bind
$nombre = "Charles";
$ciudad = "Valladolid";

$enlace_datos->bindParam(':nombre', $nombre);
$enlace_datos->bindParam(':ciudad', $ciudad);

// Execute
$enlace_datos->execute();
```

Método bindParam()

► El método `bindParam()` aceptan un **tercer parámetro**, que define el tipo de dato que se espera y también la longitud máxima de ese dato.

► Los **tipos de datos** más utilizados son:

- `PDO::PARAM_BOOL` (*booleano*)
- `PDO::PARAM_NULL` (*null*)
- `PDO::PARAM_INT` (*integer*)
- `PDO::PARAM_STR` (*string*)

```
$enlace_datos->bindParam(':edad', $edad, PDO::PARAM_INT);
```

```
$enlace_datos->bindParam(':nombre', $nombre, PDO::PARAM_STR, 12);
```



```
SELECT * FROM tabla LIMIT [OFF_SET,] N;
```

- ▶ La palabra clave **limit** se usa para limitar el número de filas devueltas en un resultado de consulta, se puede usar con select, update, delete.
- ▶ **N** es cualquier número entero que comienza desde 0, poniendo 0 el límite no devuelve ningún registro en la consulta.
- ▶ N=5 devolverá cinco registros. Si los registros en la tabla especificada son menores que N, entonces todos los registros de la tabla consultada se devuelven en el conjunto de resultados y no se producirán errores.
- ▶ El valor **OFF_SET**, número entero, nos permite especificar en qué fila(registro) comenzar la recuperación de datos. El número de registro inicial es 0 (no 1)

```
SELECT * FROM tabla LIMIT [OFF_SET,] N;
```

- La cláusula LIMIT en MySQL se utiliza para restringir el número de filas que se devuelven en una consulta. Permite especificar cuántas filas se deben devolver a partir del comienzo del conjunto de resultados.

```
SELECT columnas  
FROM tabla  
LIMIT número_de_filas;
```

número_de_filas: El número máximo de filas que deseas que se devuelvan.

- La cláusula LIMIT junto con OFFSET se utiliza para la paginación de resultados en consultas SQL. La combinación de estas dos cláusulas permite seleccionar un rango específico de filas.

```
SELECT *  
FROM productos  
ORDER BY id_producto  
LIMIT 10 OFFSET 20;
```

LIMIT 10 indica que solo se deben devolver 10 filas.

OFFSET 20 indica que se debe comenzar desde la fila número 20 (registro número 20 que cumple la condición select)

Importante:

- El primer registro que es **0**.
- La sintaxis "**LIMIT 20, 10**" en MySQL **está obsoleta** y fue reemplazada por "**LIMIT 10 offset 20**".

- ▶ El **tercer parámetro de** `bindParam()` que define el tipo de dato, es necesario cuando se utiliza la cláusula **limit** de SQL e interrogantes para los valores.
- ▶ Puesto que PHP envía el dato como un `string` y **SQL** necesita un entero.
- ▶ Sin embargo, si utilizamos nombres de variables **:nombre**, no habrá ningún problema.

```
$inicio = 0;
$cantidad = 3;
$enlace = self::$conexion->prepare('SELECT * FROM usuarios LIMIT ?,?');

//Cuidado con la cláusula limit, espera como valores números enteros (int),
php los pasa como string

//Es obligatorio especificar el tipo de datos para que realice un casting,
en caso contrario, se producirá un error

$enlace->bindParam(1, $inicio, PDO::PARAM_INT);
$enlace->bindParam(2, $cantidad, PDO::PARAM_INT);
```



```
$inicio = 0;
$cantidad = 3;
$enlace = self::$conexion->prepare('SELECT * FROM usuarios LIMIT :inicio, :cantidad);

//La cláusula limit, espera como valores números enteros (int), php los pasa como string
//Con esta sintaxis no hay ningún problema
$enlace->bindParam(':inicio', $inicio);
$enlace->bindParam(':cantidad', $cantidad);
```

PDO con la cláusula SQL LIKE

La cláusula SQL LIKE.

► Podemos pensar que una consulta de este tipo es válida:

```
$enlace = self::$conexion->prepare('SELECT nombre FROM usuarios where nombre like %?%');
```

- Esto `%?%` producirá un error.
- Para comprender el problema, hay que entender que un marcador `?` sólo puede representar un dato literal completo, es decir, una cadena o un número.
- **Y de ninguna manera puede representar una parte de un literal o alguna parte arbitraria de SQL.**
- Por lo tanto, cuando se trabaja con LIKE, **tenemos que preparar nuestro literal completo primero, y luego enviarlo a la consulta.**



```
self::$conexion = Conectar::conexion();  
$buscar = '%JUAN%'; //Muy IMPORTANTE  
//Cuidado con la cláusula like, necesita ser construida antes de enviarla a prepare()  
$enlace = self::$conexion->prepare('SELECT nombre FROM usuarios where nombre like ?');  
$enlace->bindParam(1, $buscar);
```

- ▶ Es el método `execute()` es el que realmente **envía o recoge** los datos a la base de datos.
- ▶ Si no se llama a `execute ()` no se obtendrán los resultados sino un error.

Método execute()

```
public PDOStatement::execute ( [ array $input_parameters ] ) : bool
```

- Ejecuta la **sentencia preparada** y devuelve **true o false** según el resultado de la ejecución
- Si esta sentencia incluía parámetros, se debe:
 1. O llamar a `PDOStatement::bindParam()` para vincular variables o valores (respectivamente) a los marcadores de parámetros y posteriormente al método `execute()`.
 2. O bien **pasar un array con los valores de los parámetros** al método `execute()` sin necesidad de llamar a `bindParam()`.

Método execute()

```
<?php
/* Ejecutar una sentencia preparada proporcionando un array de valores de inserción */
$calorias =150;
$color='red';
$enlace_datos =$conexion->prepare('SELECT name, colour, calories
FROM fruit WHERE calories = ? AND colour = ? ');
/*No hay método bindParam()*/
$enlace_datos->execute(array($calorias,$color));
```

Métodos PDO muy útiles

PDO::exec()

PDO::exec(). Ejecuta una sentencia SQL y devuelve el número de filas afectadas. Devuelve el número de filas insertadas, borradas o actualizadas.

No devuelve resultados de una sentencia SELECT

```
// La siguiente instrucción devuelve 1 si que se ha eliminado un registro  
  
echo $filas_afectadas = $conexion->exec("DELETE FROM Clientes WHERE nombre='Luis'");  
  
//No devuelve el número de filas si la instrucción sql es un SELECT  
//Ejemplo: Devuelve 0 si ejecutó, aunque existan registros  
  
echo $filas_afectadas = $conexion->exec("SELECT * FROM Clientes");
```

Métodos PDO

PDO::lastInsertId ()

Este método devuelve el **id autoincrementado del último registro en esa conexión**

Se debe ejecutar sobre un objeto de la clase PDO, **no** sobre un objeto de la clase PDOStatement

```
public PDO::lastInsertId ()
```

```
$enlace_datos = $conexion -> prepare("INSERT INTO usuarios (nombre) VALUES (:nombre)");  
$nombre = "Angelina";  
$enlace_datos->bindValue(':nombre', $nombre);  
$enlace_datos->execute();  
echo $id = $conexion->lastInsertId();
```


Métodos PDOStatement

public PDOStatement::rowCount (void) : int

Devuelve el número de filas afectadas por la última sentencia SQL

```
public PDOStatement::rowCount( void ) : int
```

```
$enlace_datos= $conexion->prepare("SELECT * FROM usuarios");  
$enlace_datos->execute();  
echo $numero_filas=$enlace_datos->rowCount();
```

Métodos PDOStatement

```
public PDOStatement::fetchColumn ([ int $column_number = 0 ] ) : mixed
```

Devuelve **una única columna de un conjunto de resultados**.
La columna se indica con un entero, empezando desde cero.
Si no se proporciona valor, obtiene la primera columna.

```
$enlace_datos = $conexion->prepare("SELECT * FROM Usuarios");  
$enlace_datos->execute();  
while ($columna= $enlace_datos->fetchColumn(1)) {  
    echo "Nombre: $columna <br>";  
}
```

Cerrar la conexión y liberar variables

Libera la variable que contiene los datos recuperados de la BD

/ La siguiente llamada a closeCursor() podría ser necesaria para algunos controladores */*

```
$enlace_datos->closeCursor();
```

MUY IMPORTANTE: cierra la conexión con la BD:

```
$conexion = NULL;
```

Cerrar la conexión y liberar variables

Cerrar el cursor libera los recursos asociados con el objeto PDOStatement y permite que la sentencia SQL sea ejecutada de nuevo.

1. **Reutilización de la Sentencia:** Si planeas ejecutar la misma sentencia SQL varias veces, cerrar el cursor permite que la sentencia sea ejecutada de nuevo sin problemas.
2. **Liberación de Recursos:** Cerrar el cursor libera los recursos del servidor de base de datos asociados con la consulta, lo cual es importante para la eficiencia y el rendimiento.
3. **Evitar Bloqueos:** En algunos sistemas de bases de datos, no cerrar el cursor puede llevar a bloqueos o a un uso excesivo de recursos.

Transacciones en BD

- Las transacciones son un conjunto de operaciones que se tratan como una sola unidad de trabajo.
- Si todas las operaciones se completan con éxito, los cambios se confirman y se hacen permanentes en la base de datos.
- Si alguna de las operaciones falla, todos los cambios se deshacen, y la base de datos vuelve a su estado anterior a la transacción.
- Las transacciones son importantes en las aplicaciones que requieren consistencia en la base de datos, como los sistemas bancarios, donde una operación de transferencia de dinero implica una operación de débito y una operación de crédito.
- En PHP, puedes trabajar con transacciones en MySQL utilizando los siguientes métodos del **objeto PDO**:
 - ✓ `beginTransaction()`: Inicia una transacción.
 - ✓ `commit()`: Confirma una transacción, haciendo permanentes todos los cambios en la base de datos.
 - ✓ `rollback()`: Revierte una transacción, deshaciendo todos los cambios en la base de datos.
- Debes usar `setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION)` para configurar PDO para que lance excepciones cuando ocurra un error. Esto es útil para manejar errores en las transacciones.
- En el código, inicias una transacción `beginTransaction()`, luego realizas una operación de débito y una operación de crédito, por ejemplo. Si ambas operaciones se completan con éxito, confirmas la transacción con `commit()`. Si alguna de las operaciones falla, reviertes la transacción con `rollback()`.

Transacciones en BD

Ejemplo: Creamos una base de datos Banco con una sola tabla llamada Cuentas:

```
CREATE DATABASE banco;
```

```
USE banco;
```

```
CREATE TABLE cuentas (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    saldo DECIMAL(10, 2) NOT NULL  
);
```

```
INSERT INTO cuentas (nombre, saldo) VALUES  
('Carlos', 1000.00),  
('Ana', 2000.00),  
('Luis', 3000.00),  
('María', 4000.00);
```

```

<?php
//Base datos Banco
//Tabla cuentas
$usuario = 'root';
$contrasena = '';
$moneto = 1000;
$idOrigen = 1;
$idDestino = 2;

```

```

try {
$dsn = 'mysql:host=localhost;dbname=banco;port=3306;charset=utf8mb4';

$conexion = new PDO($dsn, $usuario, $contrasena);
$conexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
// Iniciar la transacción
$conexion->beginTransaction();
// Realizar la operación de débito
$sqlDebito = 'UPDATE cuentas SET saldo = saldo - :moneto WHERE id = :id_origen';
$stmtDebito = $conexion->prepare($sqlDebito);
$stmtDebito->execute(['moneto' => $moneto, 'id_origen' => $idOrigen]);
//Verificar si la operación de débito afectó alguna fila
if ($stmtDebito->rowCount() == 0) {
    throw new PDOException('La cuenta de origen no existe');
}
// Realizar la operación de crédito
$sqlCredito = 'UPDATE cuentas SET saldo = saldo + :moneto WHERE id = :id_destino';
$stmtCredito = $conexion->prepare($sqlCredito);
$stmtCredito->execute(['moneto' => $moneto, 'id_destino' => $idDestino]);
if ($stmtCredito->rowCount() == 0) {
    throw new PDOException('La cuenta de destino no existe');
}
// Confirmar la transacción
$conexion->commit();
echo "Transacción realizada con éxito";
} catch (PDOException $e) {
    // Si algo sale mal, revertir la transacción
    $conexion->rollBack();
    echo "<br>Error:{$e->getMessage()}<br>Línea del error: {$e->getLine()} <br>Archivo del error: {$e->getFile()} ";
}

```

id	nombre	saldo
1	Carlos	1000.00
2	Ana	2000.00
3	Luis	3000.00
4	María	4000.00

Manipulación de Objetos en PDO

`FETCH_CLASS`

`FETCH_PROPS_LATE`

Métodos de Mapeo Avanzado

`setFetchMode()`

`fetchObject()`

Mapear resultados de una consulta SQL a objetos de PHP

- ▶ Al trabajar con bases de datos en PHP, PDO (PHP Data Objects) ofrece un conjunto de herramientas para **mapear resultados de una consulta SQL a objetos de PHP.**
- ▶ Esto facilita el trabajo en aplicaciones orientadas a objetos, permitiendo manipular los datos directamente como instancias de clases en lugar de arrays.
- ▶ Técnicas de mapeo de objetos en PDO:
 - `PDO::FETCH_CLASS,`
 - `PDO::FETCH_PROPS_LATE,`
 - `setFetchMode()`
 - `fetchObject()`

PDO::FETCH_CLASS

- ▶ Es una constante de PDO que configura el modo de obtención de resultados de una consulta en forma de objetos.
- ▶ Este modo de obtención **convierte cada fila de una consulta en una instancia de una clase dada, asignando automáticamente los valores de las columnas a las propiedades de la clase que coincidan en nombre.**
- ▶ Cuando se usa `PDO::FETCH_CLASS` , PDO asigna los valores de las columnas **antes** de llamar al constructor de la clase.
- ▶ Si la lógica del constructor depende de estos valores, **puede que no funcione como se espera**, pues aún no se han establecido las propiedades.

PDO::FETCH_PROPS_LATE

- ▶ Esta constante se utiliza **junto** con `PDO::FETCH_CLASS` para controlar cuándo se asignan los valores de las propiedades en la clase.

`PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE`

- ▶ Esta opción **cambia el orden** en el que PDO asigna los valores a las propiedades de la clase:
 - ❖ En vez de asignar los valores antes de llamar al constructor, como ocurre con `PDO::FETCH_CLASS`
 - ❖ `PDO::FETCH_PROPS_LATE` llama primero al constructor y luego asigna los valores, lo que permite trabajar con constructores que dependen de las propiedades inicializadas por PDO.
- ▶ Esto garantiza que cualquier lógica dependiente de las propiedades en el constructor funcione correctamente.

- Este método permite establecer un modo de obtención de datos predeterminado para el objeto PDOStatement.
- Configurando, por ejemplo, que cada fila de la consulta se devuelva como una instancia de una clase específica, sin necesidad de especificarlo en cada llamada a fetch() o fetchAll().
- Esto es útil cuando se reutiliza el mismo modo de obtención varias veces dentro de un PDOStatement.

Método setFetchMode()

```
PDOStatement::setFetchMode(PDO::FETCH_CLASS, string $classname, ?array $ctorargs = NULL)
```

```
<?php
class Usuario
{
    public $id;
    public $nombre;
    public $email;
    private $fechaCreacion;

    public function __construct()
    {
        // Inicializa la fecha de creación al momento de instanciar la clase
        $this->fechaCreacion = date("Y-m-d H:i:s");
        echo "Constructor de Usuario llamado. Fecha de creación: {$this->fechaCreacion}<br>";
    }

    public function mostrarInformacion()
    {
        // Accedemos a las propiedades ya asignadas después de la creación del objeto
        return "Usuario: $this->nombre, Email: $this->email";
    }
}

// Configuración de PDO (suponiendo que ya tienes una conexión a la base de datos en $pdo)
$stmt = $pdo->query("SELECT id, nombre, email FROM usuarios");
$stmt->setFetchMode(PDO::FETCH_CLASS, 'Usuario');

// Recorrido de cada fila de la consulta y creación de objetos Usuario
while ($usuario = $stmt->fetch()) {
    // El objeto $usuario es una instancia de Usuario con las propiedades ya asignadas
    echo $usuario->mostrarInformacion() . "<br>";
}
?>
```

```
<?php
class Usuario
{
    public $id;
    public $nombre;
    public $email;

    public function __construct()
    {
        echo "Constructor de Usuario llamado.<br>";
        if ($this->nombre) {
            echo "Nombre del usuario: $this->nombre<br>";
        } else {
            echo "Propiedad 'nombre' aún no asignada.<br>";
        }
    }
}

// Ejecución de la consulta y configuración del modo de obtención con PROPS_LATE
$stmt = $pdo->query("SELECT id, nombre, email FROM usuarios");
$stmt->setFetchMode(PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE, 'Usuario');

// Fetch de cada fila y asignación a objetos de tipo Usuario
while ($usuario = $stmt->fetch()) {
    // El constructor se llama antes de asignar las propiedades
    echo "Usuario: $usuario->nombre ($usuario->email)<br>";
}
?>
```

```
class Usuario
{
    public $id;
    public $nombre;
    public $email;
    private $rol;

    public function __construct($rol)
    {
        // Se inicializa el rol del usuario
        $this->rol = $rol;
        echo "Constructor de Usuario llamado. Rol asignado: $this->rol<br>";
    }

    public function mostrarInformacion()
    {
        // Muestra la información del usuario
        return "Usuario: $this->nombre, Email: $this->email, Rol: $this->rol";
    }
}

// Suponemos que tienes una tabla 'usuarios' y quieres asignar un rol por defecto
$rolPorDefecto = 'Usuario Regular';

// Consulta a la base de datos
$consulta = $pdo->query("SELECT id, nombre, email FROM usuarios");
$consulta->setFetchMode(PDO::FETCH_CLASS, 'Usuario', [$rolPorDefecto]);

// Recorrido de cada fila de la consulta y creación de objetos Usuario
while ($usuario = $consulta->fetch()) {
    // El objeto $usuario es una instancia de Usuario con las propiedades ya asignadas
    echo $usuario->mostrarInformacion() . "<br>";
}
```

Reglas de Asignación de Propiedades

PDO::FETCH_CLASS | PDO::FETCH_PROPS_LATE

1. Atributos Existentes:

- Si la clase tiene un atributo cuyo nombre coincide con el de una columna, el valor de la columna se asignará a ese atributo.

2. Método Mágico `__set`:

- Si no existe tal atributo, se llamará al método mágico `__set()` (si está definido) y se ejecutarán las instrucciones allí indicadas.

3. Creación de Propiedades Públicas:

- Si el método `__set()` no está definido, se creará una propiedad pública en la clase y se le asignará el valor de la columna.

4. Evitar la Creación de Propiedades:

- Si deseas que una propiedad no existente en la clase no sea asignada al objeto, puedes definir un método `__set()` vacío:

```
public function __set($name, $value) {}
```

5. Asignación a Propiedades Privadas

- PDO puede asignar valores a las propiedades privadas de una clase, lo cual es sorprendente pero útil.

6. Orden de Asignación y Constructor

- **Por Defecto:** PDO asigna los valores a los atributos de la clase antes de llamar al constructor.
- **Modificar comportamiento:** Para cambiar este comportamiento y asignar los valores después de llamar al constructor, se utiliza `PDO::FETCH_PROPS_LATE`.


```

<?php
class Usuario
{
    public $nombre;
    public $email;
    private $edad;

    public function __construct()
    {
        echo "Constructor llamado.<br>";
    }

    public function __set($name, $value)
    {
        // Asignar el valor a una propiedad pública o privada o a una propiedad que no existe
        $this->$name = $value;
    }

    public function mostrarInformacion()
    {
        return "Nombre: $this->nombre, Email: $this->email, Edad: $this->edad";
    }
}

// Crear una instancia de Usuario
$usuario = new Usuario();
$usuario->nombre = "Juan";
$usuario->email = "juan@example.com";
$usuario->edad = 30; // Esto llamará al método __set

echo $usuario->mostrarInformacion() . "<br>";

// Asignar una nueva propiedad usando __set
$usuario->rol = "Administrador"; // Esto llamará al método __set
echo "Rol: " . $usuario->rol . "<br>";
var_dump($usuario);

```

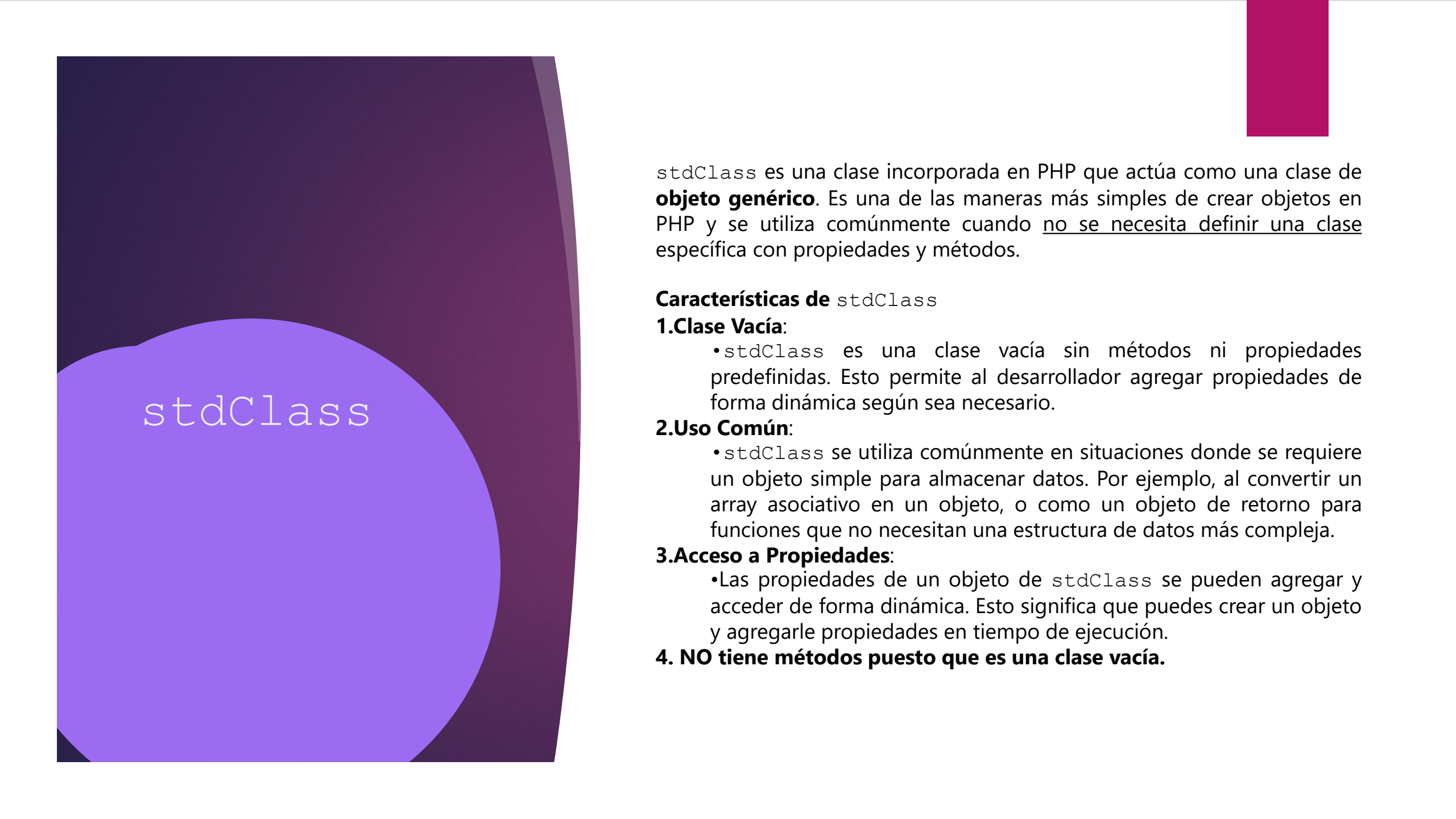
Constructor llamado.
 Nombre: Juan, Email: juan@example.com, Edad: 30
 Rol: Administrador

C:\xampp\htdocs\xClase\asd\base de datos\borrar.php:36:
 object(Usuario)[1]
 public 'nombre' => string 'Juan' (length=4)
 public 'email' => string 'juan@example.com' (length=16)
 private 'edad' => int 30
 public 'rol' => string 'Administrador' (length=13)

- Este método permite obtener **una sola fila de resultados** de una consulta y devolverla como un objeto de una clase específica o como un **objeto stdClass** si no se especifica una clase.
- `$class` (opcional): Especifica el nombre de la clase que se debe instanciar para cada fila obtenida. Si no se proporciona, `fetchObject()` devolverá un objeto de la clase `stdClass`, que es un objeto anónimo sin métodos específicos.
- `$constructorArgs` (opcional): Es un array con los argumentos que se pasarán al constructor de la clase especificada en `$class`. Si no se pasa, el constructor no recibirá argumentos.
- No permite asignar las propiedades de la clase antes de llamar al constructor.

Método PDOStatement::fetchObject()

```
public PDOStatement::fetchObject(?string $class = "stdClass", array $constructorArgs = []): object|false
```



stdClass

`stdClass` es una clase incorporada en PHP que actúa como una clase de **objeto genérico**. Es una de las maneras más simples de crear objetos en PHP y se utiliza comúnmente cuando no se necesita definir una clase específica con propiedades y métodos.

Características de `stdClass`

1. Clase Vacía:

- `stdClass` es una clase vacía sin métodos ni propiedades predefinidas. Esto permite al desarrollador agregar propiedades de forma dinámica según sea necesario.

2. Uso Común:

- `stdClass` se utiliza comúnmente en situaciones donde se requiere un objeto simple para almacenar datos. Por ejemplo, al convertir un array asociativo en un objeto, o como un objeto de retorno para funciones que no necesitan una estructura de datos más compleja.

3. Acceso a Propiedades:

- Las propiedades de un objeto de `stdClass` se pueden agregar y acceder de forma dinámica. Esto significa que puedes crear un objeto y agregarle propiedades en tiempo de ejecución.

4. NO tiene métodos puesto que es una clase vacía.

stdClass

```
<?php
$obj = new stdClass();
$obj->nombre = "Juan";
$obj->email = "juan@example.com";
$obj->edad = 30;

echo "Nombre: " . $obj->nombre . "<br>";
echo "Email: " . $obj->email . "<br>";
echo "Edad: " . $obj->edad . "<br>";
?>
```

```
<?php
$array = [
    "nombre" => "Maria",
    "email" => "maria@example.com",
    "edad" => 25
];

$obj = (object) $array;

echo "Nombre: " . $obj->nombre . "<br>";
echo "Email: " . $obj->email . "<br>";
echo "Edad: " . $obj->edad . "<br>";

?>
```

```
<?php
function crearUsuario($nombre, $email, $edad)
{
    $usuario = new stdClass();
    $usuario->nombre = $nombre;
    $usuario->email = $email;
    $usuario->edad = $edad;
    return $usuario;
}

$usuario = crearUsuario("Carlos", "carlos@example.com", 40);
echo "Nombre: " . $usuario->nombre . "<br>";
echo "Email: " . $usuario->email . "<br>";
echo "Edad: " . $usuario->edad . "<br>";

?>
```

```

<?php
class Usuario
{
    public $nombre;
    public $email;
    private $id;

    public function __construct($nombre = null, $email = null, $id = null)
    {
        $this->nombre = $nombre;
        $this->email = $email;
        $this->id = $id;
        echo "Constructor llamado.<br>";
    }

    public function mostrarInformacion()
    {
        return "Nombre: $this->nombre, Email: $this->email, Id: $this->id";
    }
}

try {
    $pdo = new PDO('mysql:host=localhost;dbname=probando', 'root', '');
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmt = $pdo->query("SELECT nombre, email, id FROM usuarios LIMIT 1");
    $usuario = $stmt->fetchObject();

    echo "Nombre: " . $usuario->nombre . "<br>";
    echo "Email: " . $usuario->email . "<br>";
    echo "Id: " . $usuario->id . "<br>";

    var_dump($usuario);
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}

```

Nombre: Juan Perez
 Email: juan.perez@example.com
 Id: 1

C:\xampp\htdocs\xClase\asd\base de datos\borrar.php:32:

```

object(stdClass)[3]
  public 'nombre' => string 'Juan Perez' (length=10)
  public 'email' => string 'juan.perez@example.com' (length=22)
  public 'id' => int 1

```

```

<?php
class Usuario
{
    public $nombre;
    public $email;
    private $id;

    public function __construct($nombre = null, $email = null, $id = null)
    {
        $this->nombre = $nombre;
        $this->email = $email;
        $this->id = $id;
        echo "Constructor llamado.<br>";
    }

    public function mostrarInformacion()
    {
        return "Nombre: $this->nombre, Email: $this->email, Id: $this->id";
    }
}

try {
    $pdo = new PDO('mysql:host=localhost;dbname=probando', 'root', '');
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmt = $pdo->query("SELECT nombre, email, id FROM usuarios LIMIT 1");
    $usuario = $stmt->fetchObject('Usuario');

    echo $usuario->mostrarInformacion() . "<br>";
    var_dump($usuario);
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}

```

Constructor llamado.

Nombre: , Email: , Id:

C:\xampp\htdocs\xClaSe\asd\base de datos\borrar.php:30:

object(Usuario)[3]

public 'nombre' => null

public 'email' => null

private 'id' => null

```

<?php
class Usuario
{
    public $nombre;
    public $email;
    private $id;

    public function __construct()
    {
        echo "Constructor llamado.<br>";
    }

    public function mostrarInformacion()
    {
        return "Nombre: $this->nombre, Email: $this->email, Id: $this->id";
    }
}

try {
    $pdo = new PDO('mysql:host=localhost;dbname=probando', 'root', '');
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmt = $pdo->query("SELECT nombre, email, id FROM usuarios LIMIT 1");

    //fetchObject('Usuario') asignará las propiedades de la fila de resultados a las propiedades de la clase Usuario.
    $usuario = $stmt->fetchObject('Usuario');

    echo $usuario->mostrarInformacion() . "<br>";
    var_dump($usuario);
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}

```

Constructor llamado.

Nombre: Juan Perez, Email: juan.perez@example.com, Id: 1

C:\xampp\htdocs\xClase\asd\base de datos\borrar.php:28:

object(Usuario)[3]

public 'nombre' => string 'Juan Perez' (length=10)

public 'email' => string 'juan.perez@example.com' (length=22)

private 'id' => int 1